

计算机考研复试面试常问问题 数据结构篇

计算机考研复试面试常问问题 数据结构篇

第一章、绪论

第二章、线性表

第三章、栈和队列

第四章、串

第五章、树与二叉树

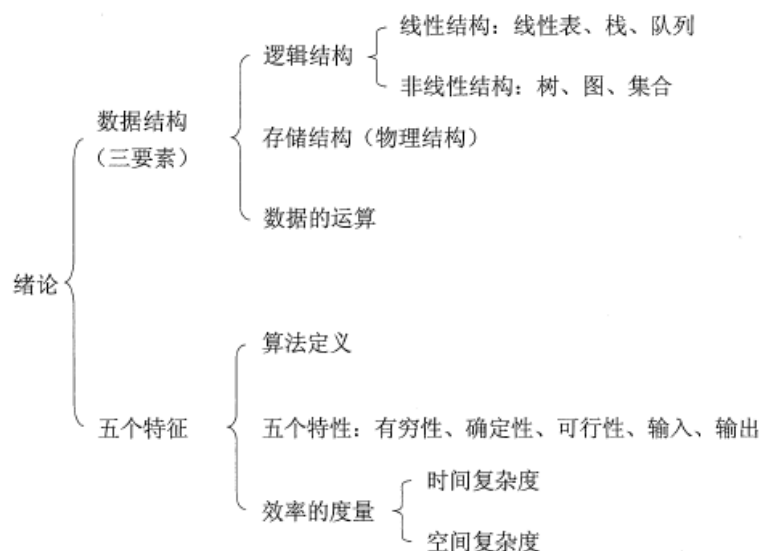
第六章、图

第七章、查找

第八章、排序

第一章、绪论

快速唤起记忆知识框架：



1. 时间复杂度

- 1 | **ps:**我看到一个学长回忆说老师问了他 大O 是什么意思???如果是我, 估计当场蒙蔽了, 毕竟平时知道做题就行, 也没仔细看 大O 什么意思, 所以大家还是看看这题。有备无患。

一个语句的频度是指该语句在算法中被重复执行的次数。算法中所有语句的频度之和记为 $T(n)$, 它是该算法问题规模 n 的函数, 时间复杂度主要分析 $T(n)$ 的数量级。算法中基本运算(最深层循环内的语句)的频度与 $T(n)$ 同数量级, 因此通常采用算法中基本运算的频度 $f(n)$ 来分析算法的时间复杂度。因此, 算法的时间复杂度记为 $T(n) = O(f(n))$

取 $f(n)$ 中随 n 增长最快的项, 将其系数置为1作为时间复杂度的度量。例如, $f(n) = an^3 + bn^2 + cn$ 的时间复杂度为 $O(n^3)$

上式中, O 的含义是 $T(n)$ 的数量级, 其严格的数学定义是: 若 $T(n)$ 和 $f(n)$ 是定义在正整数集合上的两个函数, 则存在正常数 C 和 n_0 , 使得当 $n \geq n_0$ 时, 都满足 $0 \leq T(n) \leq Cf(n)$ 。

算法的时间复杂度不仅依赖于问题的规模 n , 也取决于待输入数据的性质(如输入数据元素的初始状态)

2. 空间复杂度

算法的空间复杂度 $S(n)$ 定义为该算法所耗费的存储空间, 它是问题规模 n 的函数。记为

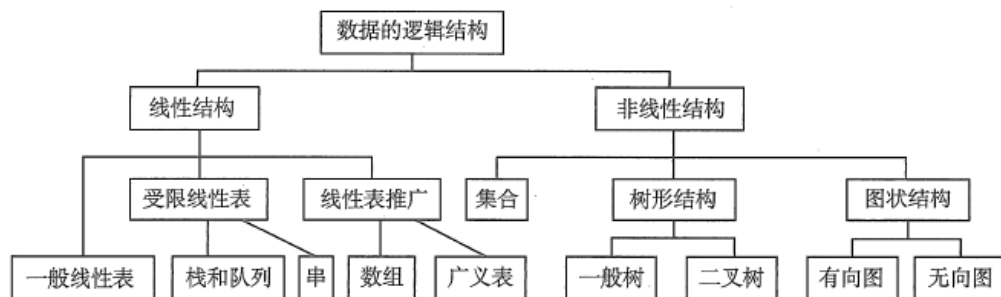
$$S(n) = O(g(n))$$

一个程序在执行时除需要存储空间来存放本身所用的指令、常数、变量和输入数据外, 还需要一些对数据进行操作的工作单元和存储一些为实现计算所需信息的辅助空间。若输入数据所占空间只取决于问题本身, 和算法无关, 则只需分析除输入和程序之外的额外空间。

算法原地工作是指算法所需的辅助空间为常量, 即 $O(1)$ 。

3. 数的逻辑结构

指的是数据元素之间逻辑关系, 与数的存储结构无关, 是独立于计算机的, 以下是分类图。



4. 数的存储结构

存储结构是指数据结构在计算机中的表示，也称物理结构，主要有以下4种：

- 1) **顺序存储**。把逻辑上相邻的元素存储在物理位置上也相邻的存储单元中，元素之间的关系由存储单元的邻接关系来体现。其优点是可实现随机存取，每个元素占用最少的存储空间；缺点是只能使用相邻的一整块存储单元，因此可能产生较多的外部碎片。
- 2) **链式存储**。不要求逻辑上相邻的元素在物理位置上也相邻，借助指示元素存储地址的指针来表示元素之间的逻辑关系。其优点是不会出现碎片现象，能充分利用所有存储单元；缺点是每个元素因存储指针而占用额外的存储空间，且只能实现顺序存取。
- 3) **索引存储**。在存储元素信息的同时，还建立附加的索引表。索引表中的每项称为索引项，索引项的一般形式是（关键字，地址）。其优点是检索速度快；缺点是附加的索引表额外占用存储空间。另外，增加和删除数据时也要修改索引表，因而会花费较多的时间。
- 4) **散列存储**。根据元素的关键字直接计算出该元素的存储地址，又称哈希(Hash)存储。其优点是检索、增加和删除结点的操作都很快；缺点是若散列函数不好，则可能出现元素存储单元的冲突，而解决冲突会增加时间和空间开销。

5. 用循环比递归的效率高吗？

循环和递归两者是可以互换的，不能决定性的说循环的效率比递归高。

递归的优点是：代码简洁清晰，容易检查正确性；缺点是：当递归调用的次数较多时，要增加额外的堆栈处理，有可能产生堆栈溢出的情况，对执行效率有一定的影响。

循环的优点是：结构简单，速度快；缺点是：它并不能解决全部问题，有的问题适合于用递归来解决不适合用循环。

6. 贪心算法和动态规划以及分治法的区别？

贪心算法顾名思义就是做出在当前看来是最好的结果，它不从整体上加以考虑，也就是局部最优解。贪心算法从上往下，从顶部一步一步最优，得到最后的结果，它不能保证全局最优解，与贪心策略的选择有关。

动态规划是把问题分解成子问题，这些子问题可能有重复，可以记录下前面子问题的结果防止重复计算。动态规划解决子问题，前一个子问题的解对后一个子问题产生一定的影响。在求解子问题的过程中保留哪些有可能得到最优的局部解，丢弃其他局部解，直到解决最后一个问题时也就是初始问题的解。动态规划是从下到上，一步一步找到全局最优解。（各子问题重叠）

分治法 (divide-and-conquer)：将原问题划分成n个规模较小而结构与原问题相似的子问题；递归地解决这些子问题，然后再合并其结果，就得到原问题的解。（各子问题独立）

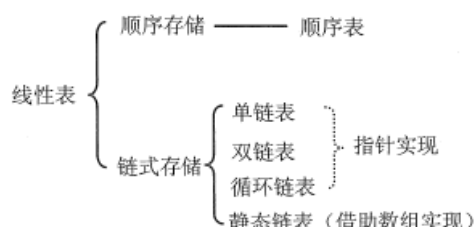
分治模式在每一层递归上都有三个步骤：

分解 (Divide) : 将原问题分解成一系列子问题;
解决 (conquer) : 递归地解各个子问题。若子问题足够小, 则直接求解;
合并 (Combine) : 将子问题的结果合并成原问题的解。

例如归并排序。

第二章、线性表

快速唤起记忆知识框架:



7. 顺序表和链表的比较

1. 存取 (读写) 方式

顺序表可以顺序存取, 也可以随机存取, 链表只能从表头顺序存取元素。例如在第*i*个位置上执行存取的操作, 顺序表仅需一次访问, 而链表则需从表头开始依次访问*i*次。

2. 逻辑结构与物理结构

采用顺序存储时, 逻辑上相邻的元素, 对应的物理存储位置也相邻。而采用链式存储时, 逻辑上相邻的元素, 物理存储位置则不一定相邻, 对应的逻辑关系是通过指针链接来表示的。

3. 查找、插入和删除操作

对于按值查找, 顺序表无序时, 两者的时间复杂度均为 $O(n)$; 顺序表有序时, 可采用折半查找, 此时的时间复杂度为 $O(\log_2 n)$ 。

对于按序号查找, 顺序表支持随机访问, 时间复杂度仅为 $O(1)$, 而链表的平均时间复杂度为 $O(n)$ 。顺序表的插入、删除操作, 平均需要移动半个表长的元素。链表的插入、删除操作, 只需修改相关结点的指针域即可。由于链表的每个结点都带有指针域, 故而存储密度不够大。

4. 空间分配

顺序存储在静态存储分配情形下, 一旦存储空间装满就不能扩充, 若再加入新元素, 则会出现内存溢出, 因此需要预先分配足够大的存储空间。预先分配过大, 可能会导致顺序表后部大量闲置; 预先分配过小, 又会造成溢出。动态存储分配虽然存储空间可以扩充, 但需要移动大量元素, 导致操作效率降低, 而且若内存中没有更大块的连续存储空间, 则会导致分配失败。链式存储的结点空间只在需要时申请分配, 只要内存有空间就可以分配, 操作灵活、高效。

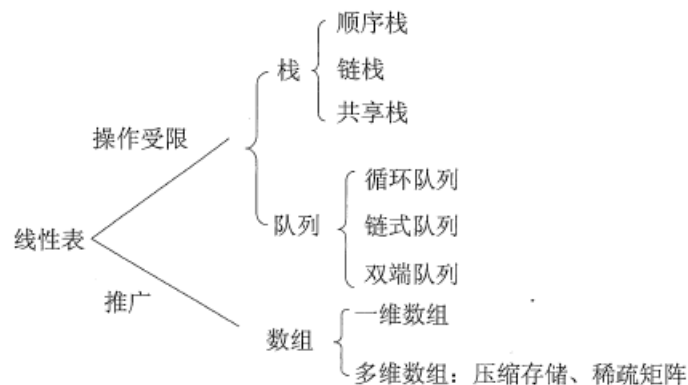
8. 头指针和头结点的区别?

头指针: 是指向第一个节点存储位置的指针, 具有标识作用, 头指针是链表的必要元素, 无论链表是否为空, 头指针都存在。

头结点: 是放在第一个元素节点之前, 便于在第一个元素节点之前进行插入和删除的操作, 头结点不是链表的必须元素, 可有可无, 头结点的数据域也可以不存储任何信息。

第三章、栈和队列

快速唤起记忆知识框架：



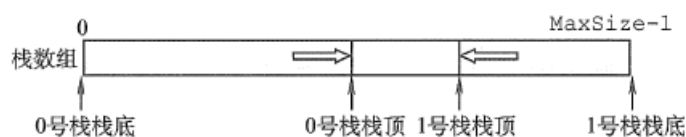
9. 栈和队列的区别？

队列是允许在一段进行插入另一端进行删除的线性表。队列顾名思义就像排队一样，对于进入队列的元素按“先进先出”的规则处理，在表头进行删除在表尾进行插入。由于队列要进行频繁的插入和删除，一般为了高效，选择用定长数组来存储队列元素，在对队列进行操作之前要判断队列是否为空或是否已满。如果想要动态长度也可以用链表来存储队列，这时要记住队头和对位指针的地址。

栈是只能在表尾进行插入和删除操作的线性表。对于插入到栈的元素按“后进先出”的规则处理，插入和删除操作都在栈顶进行，与队列类似一般用定长数组存储栈元素。由于进栈和出栈都是在栈顶进行，因此要有一个size变量来记录当前栈的大小，当进栈时size不能超过数组长度，size+1，出栈时栈不为空，size-1。

10. 共享栈

利用栈底位置相对不变的特性，可以让两个顺序栈共享一个一维数组空间，将两个栈的栈底分别设置在共享空间的两端，两个栈顶向共享空间的中间延伸。这样能够更有效的利用存储空间，两个栈的空间相互调节，只有在整个存储空间被占满时才发生上溢。



11. 如何区分循环队列是队空还是队满？

普通情况下，循环队列队空和队满的判定条件是一样的，都是 $Q.front == Q.rear$ 。

ps: 队头指针指向第一个数；队尾指针指向最后一个数的下一个位置，即将要入队的位置。

方法一：牺牲一个单元来区分队空和队满，这个时候 $(Q.rear+1) \% MaxSize == Q.front$ 才是队满标志。

方法二：类型中增设表示元素个数的数据成员。这样，队空的条件为 $Q.size == 0$ ；队满的条件为 $Q.size == MaxSize$ 。

12. 栈在括号匹配中的算法思想？

括号匹配算法思想

- (1) 出现的凡是“左括号”，则进栈；
- (2) 出现的是“右括号”，
- 首先检查栈是否空？
- 若栈空，则表明该“右括号”多余
- 否则和栈顶元素比较？
- 若相匹配，则栈顶“左括号出栈”
- 否则表明不匹配
- (3) 表达式检验结束时，
- 若栈空，则表明表达式中匹配正确
- 否则表明“左括号”有余；

13.栈在通过后缀表达式求值的算法思想？

顺序扫描表达式的每一项，然后根据它的类型做如下相应操作：若该项是操作数，则将其压入栈中；若该项是操作符，则连续从栈中退出两个操作数 y 和 x ，形成运算指令 XY ，并将计算结果重新压入栈中。当表达式的所有项都扫描并处理完后，栈顶存放的就是最后的计算结果。

表 3.1 后缀表达式 $ABCD-*+EF/-$ 求值的过程

步	扫描项	项类型	动 作	栈中内容
1			置空栈	空
2	A	操作数	进栈	A
3	B	操作数	进栈	A B
4	C	操作数	进栈	A B C
5	D	操作数	进栈	A B C D
6	-	操作符	D、C 退栈，计算 $C-D$ ，结果 R_1 进栈	A B R_1
7	*	操作符	R_1 、B 退栈，计算 $B\times R_1$ ，结果 R_2 进栈	A R_2
8	+	操作符	R_2 、A 退栈，计算 $A+R_2$ ，结果 R_3 进栈	R_3
9	E	操作数	进栈	R_3 E
10	F	操作数	进栈	R_3 E F
11	/	操作符	F、E 退栈，计算 E/F ，结果 R_4 进栈	R_3 R_4
12	-	操作符	R_4 、 R_3 退栈，计算 R_3-R_4 ，结果 R_5 进栈	R_5

14.栈在递归中的应用？

递归是一种重要的程序设计方法。简单地说，若在一个函数、过程或数据结构的定义中又应用了它自身，则这个函数、过程或数据结构称为是递归定义的，简称递归。

它通常把一个大型的复杂问题层层转化为一个与原问题相似的规模较小的问题来求解，递归策略只需少量的代码就可以描述出解题过程所需要的多次重复计算，大大减少了程序的代码量。但在通常情况下，它的效率并不是太高。

将递归算法转换为非递归算法，通常需要借助栈来实现这种转换。

15.队列在层次遍历中的作用？

在信息处理中有一大类问题需要逐层或逐行处理。这类问题的解决方法往往是在处理当前层或当前行时就对下一层或下一行做预处理，把处理顺序安排好，待当前层或当前行处理完毕，就可以处理下一层或下一行。使用队列是为了保存下一步的处理顺序。下面用二叉树层次遍历的例子，说明队列的应用。

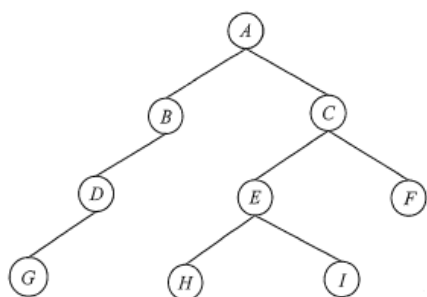


图 3.17 二叉树

序	说 明	队内	队 外
1	A 入	A	
2	A 出, BC 入	BC	A
3	B 出, D 入	CD	AB
4	C 出, EF 入	DEF	ABC
5	D 出, G 入	EFG	ABCD
6	E 出, HI 入	FGHI	ABCDE
7	F 出	GHI	ABCDEF
8	GHI 出		ABCDEFGHI

16.队列在计算机系统中的应用？

队列在计算机系统中的应用非常广泛，以下仅从两个方面来简述队列在计算机系统中的作用：第一个方面是解决主机与外部设备之间速度不匹配的问题，第二个方面是解决由多用户引起的资源竞争问题。

对于第一个方面，仅以主机和打印机之间速度不匹配的问题为例做简要说明。主机输出数据给打印机打印，输出数据的速度比打印数据的速度要快得多，由于速度不匹配，若直接把输出的数据送给打印机打印显然是不行的。解决的方法是设置一个打印数据缓冲区，主机把要打印输出的数据依次写入这个缓冲区，写满后就暂停输出，转去做其他的事情。打印机就从缓冲区中按照先进先出的原则依次取出数据并打印，打印完后再向主机发出请求。主机接到请求后再向缓冲区写入打印数据。这样做既保证了打印数据的正确，又使主机提高了效率。由此可见，打印数据缓冲区中所存储的数据就是一个队列。

对于第二个方面，CPU（即中央处理器，它包括运算器和控制器）资源的竞争就是一个典型的例子。在一个带有多终端的计算机系统上，有多个用户需要CPU 各自运行自己的程序，它们分别通过各自的终端向操作系统提出占用CPU 的请求。操作系统通常按照每个请求在时间上的先后顺序，把它们排成一个队列，每次把CPU 分配给队首请求的用户使用。当相应的程序运行结束或用完规定的时间间隔后，令其出队，再把CPU 分配给新的队首请求的用户使用。这样既能满足每个用户的请求，又使CPU 能够正常运行。

17.矩阵的压缩存储

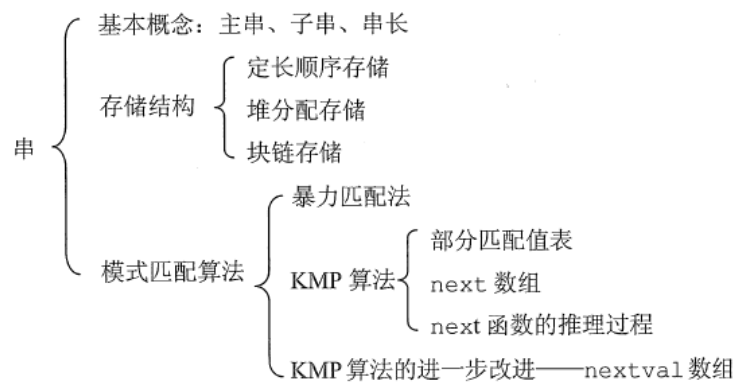
数据结构中，提供针对某些特殊矩阵的压缩存储结构。这里所说的特殊矩阵，主要分为以下两类：

- 含有大量相同数据元素的矩阵，比如对称矩阵；
- 含有大量 0 元素的矩阵，比如稀疏矩阵、上（下）三角矩阵；

针对以上两类矩阵，数据结构的压缩存储思想是：矩阵中的相同数据元素（包括元素 0）只存储一个。

第四章、串

快速唤起记忆知识框架：

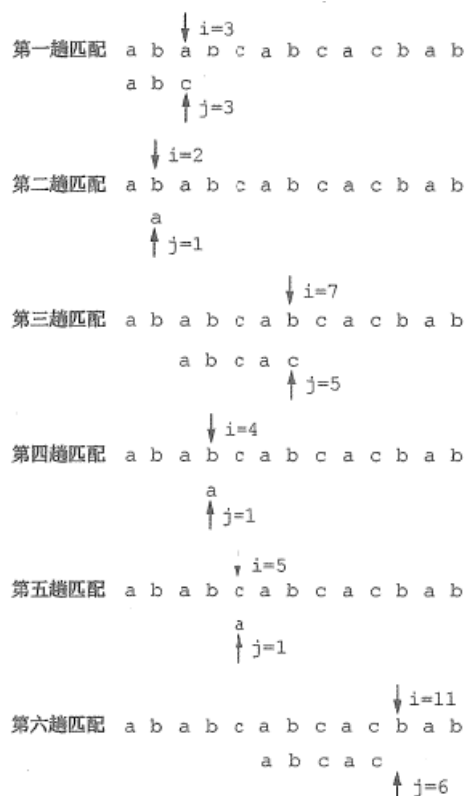


18. 串的模式匹配

子串的定位操作通常称为串的模式匹配，他求的是子串（常称模式串）在主串中的位置。

暴力模式匹配算法的思想是：从主串的第一个字符起，与子串的第一个字符比较，相等则继续比较；不等则从主串的下一个位置起，继续和子串开始比较，直到最后看是否匹配成功。

以下的子串为：'abcac'：



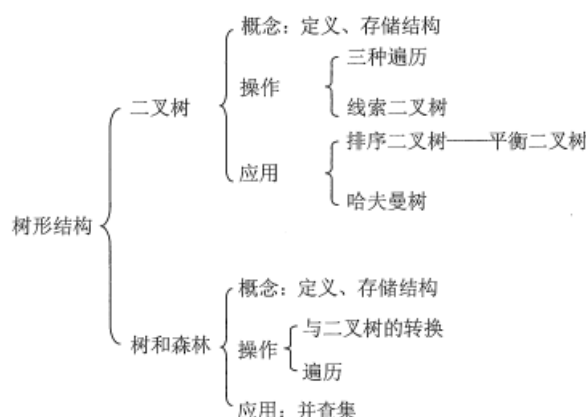
改进的模式匹配算法-----KMP算法：

在暴力匹配中，每趟匹配失败都是模式后移一位再从头开始比较。而某趟已匹配相等的字符序列是模式的某个前缀，这种频繁的重复比较相当于模式串在不断地进行自我比较，这就是其低效率的根源。因此，可以从分析模式本身的结构着手，如果已匹配相等的前缀序列中有某个后缀正好是模式的前缀，那么就可以将模式向后滑动到与这些相等字符对齐的位置，主串i指针无须回溯，并继续从该位置开始进行比较。而模式向后滑动位数的计算仅与模式本身的结构有关，与主串无关。

（此处篇幅过长，感兴趣可以直接百度KPM算法详细了解，当然要求不高的可以直接跳过，知道大致思想即可，不要被问到一问三不知就可以）

第五章、树与二叉树

快速唤起记忆知识框架：



19. 树与二叉树的相关概念？

树是非线性结构，其元素之间有明显的层次关系。在树的结构中，每个节点都只有一个前件称为父节点，没有前件的节点为树的根节点，简称为树的根；每个节点可以有多个后件成为节点的子节点，没有后件的节点称为叶子节点。

在树的结构中，一个节点所拥有的子节点个数称为该节点的度，树中最大的节点的度为树的度，树的最大的层次称为树的深度

二叉树：二叉树是另一种树形结构，其特点是每个结点至多只有两棵子树，并且二叉树的子树有左右之分，其次序不能任意颠倒。与树相似，二叉树也以递归的形式定义。二叉树是 n ($n \geq 0$) 个结点的有限集合：

- 1) 或者为空二叉树，即 $n=0$ 。
- 2) 或者由一个根结点和两个互不相交的被称为根的左子树和右子树组成。左子树和右子树又分别是一棵二叉树。

二叉树是有序树，若将其左、右子树颠倒，则成为另一棵不同的二叉树。即使树中结点只有一棵子树，也要区分它是左子树还是右子树

满二叉树：满二叉树是指除了最后一层外其他节点均有两颗子树。

完全二叉树：完全二叉树是指除了最后一层外，其他任何一层的节点数均达到最大值，且最后一层也只是在最右侧缺少节点

二叉树的存储：二叉树可以用链式存储结构来存储，满二叉树和完全二叉树可以用顺序存储结构来存储

二叉树的遍历：二叉树有先序遍历（根左右），中序遍历（左根右）和后续遍历（左右根）；还有层次遍历，需要借助一个队列。

三种遍历算法中，递归遍历左、右子树的顺序都是固定的，只是访问根结点的顺序不同。不管采用哪种遍历算法，每个结点都访问一次且仅访问一次，故时间复杂度都是 $O(n)$ 。在递归遍历中，递归工作栈的栈深恰好为树的深度，所以在最坏情况下，二叉树是有 n 个结点且深度为 n 的单支树，遍历算法的空间复杂度为 $O(n)$ 。

20. 如何由遍历序列构造一棵二叉树？

- 1) 由二叉树的先序序列和中序序列可以唯一地确定一棵二叉树。

在先序遍历序列中，第一个结点一定是二叉树的根结点；而在中序遍历中，根结点必然将中序序列分割成两个子序列，前一个子序列是根结点的左子树的中序序列，后一个子序列是根结点的右子树的中序序列。根据这两个子序列，在先序序列中找到对应的左子序列和右子序列。在先序序列中，左子序列的第一个结点是左子树的根结点，右子序列的第一个结点是右子树的根结点。如此递归地进行下去，便能唯一地确定这棵二叉树。

2)由二叉树的**后序序列和中序序列**也可以唯一地确定一棵二叉树。
因为后序序列的最后一个结点就如同先序序列的第一个结点，可以将中序序列分割成两个子序列，然后采用类似的方法递归地进行划分，进而得到一棵二叉树。

3) 由二叉树的**层序序列和中序序列**也可以唯一地确定一棵二叉树。需要注意的是，**若只知道二叉树的先序序列和后序序列，则无法唯一确定一棵二叉树。**

21.线索二叉树的概念？

对于n个结点的二叉树，在二叉链存储结构中有n+1个空链域，利用这些空链域存放在某种遍历次序下该结点的前驱结点和后继结点的指针，这些指针称为线索，加上线索的二叉树称为线索二叉树。

这种加上了线索的二叉链表称为线索链表，相应的二叉树称为线索二叉树(Threaded Binary Tree)。根据线索性质的不同，线索二叉树可分为前序线索二叉树、中序线索二叉树和后序线索二叉树三种。

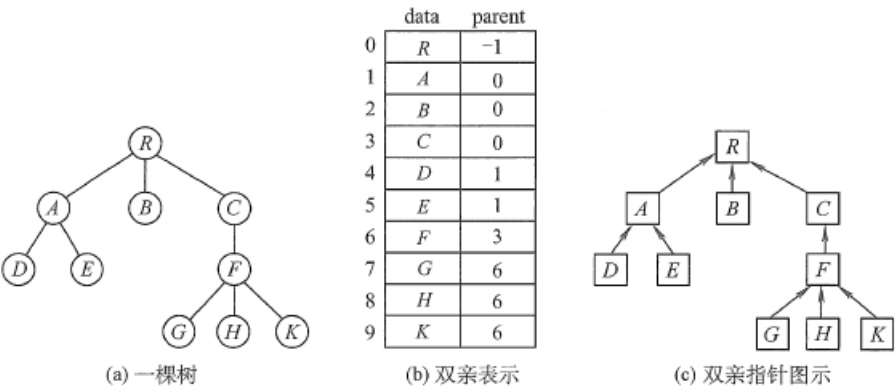
注意：线索链表解决了无法直接找到该结点在某种遍历序列中的前驱和后继结点的问题，解决了二叉链表找左、右孩子困难的问题。

二叉树的遍历本质上是将一个复杂的非线性结构转换为线性结构，使每个结点都有了唯一前驱和后继（第一个结点无前驱，最后一个结点无后继）。对于二叉树的一个结点，查找其左右子女是方便的，其前驱后继只有在遍历中得到。为了容易找到前驱和后继，有两种方法。一是在结点结构中增加向前和向后的指针，这种方法增加了存储开销，不可取；二是利用二叉树的空链指针。

22.树的存储结构？

1.双亲表示法：

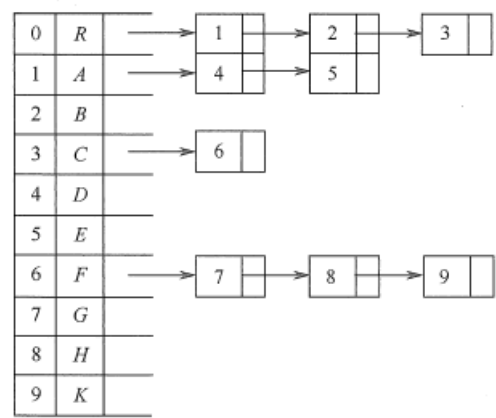
这种存储方式采用一组连续空间来存储每个结点，同时在每个结点中增设一个伪指针，指示其双亲结点在数组中的位置。



该存储结构利用了每个结点（根结点除外）只有唯一双亲的性质，可以很快得到每个结点的双亲结点，但求结点的孩子时需要遍历整个结构。

2.孩子表示法：

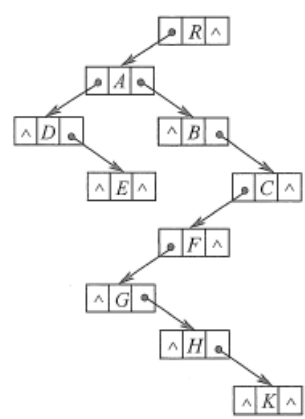
孩子表示法是将每个结点的孩子结点都用单链表链接起来形成一个线性结构，此时n 个结点就有n 个孩子链表（叶子结点的孩子链表为空表），这种存储方式寻找子女的操作非常直接，而寻找双亲的操作需要遍历n个结点中孩子链表指针域所指向的n个孩子链表。



(a) 孩子表示法

3.孩子兄弟表示法:

孩子兄弟表示法又称二叉树表示法，即以二叉链表作为树的存储结构。孩子兄弟表示法使每个结点包括三部分内容：结点值、指向结点第一个孩子结点的指针，及指向结点下一个兄弟结点的指针（沿此域可以找到结点的所有兄弟结点）



(b) 孩子兄弟表示法

这种存储表示法比较灵活，其最大的优点是可以方便地实现树转换为二叉树的操作，易于查找结点的孩子等，但缺点是从当前结点查找其双亲结点比较麻烦。若为每个结点增设一个parent域指向其父结点，则查找结点的父结点也很方便。

23.二叉排序树

1.二叉排序树的定义:

二叉排序树（也称二叉查找树）或者是一棵空树，或者是具有下列特性的二叉树：

- 1) 若左子树非空，则左子树上所有结点的值均小于根结点的值。
- 2) 若右子树非空，则右子树上所有结点的值均大于根结点的值。
- 3) 左、右子树也分别是一棵二叉排序树。

根据二叉排序树的定义，左子树结点值 < 根结点值 < 右子树结点值，所以对二叉排序树进行中序遍历，可以得到一个递增的有序序列。

2.二叉排序树的查找:

二叉排序树的查找是从根节点开始的，延某个分支逐层向下比较的过程。若二叉树非空，先将给定值与根结点的关键字比较，若相等，则查找成功；若不等，如果小于根结点的关键字，则在根结点的左子树上查找，否则在根的右子树上查找。这显然是一个递归的过程。

24. 平衡二叉树

为避免树的高度增长过快，降低二叉排序树的性能，规定在插入和删除二叉树结点时，要保证任意结点的左、右子树高度差的绝对值不超过1，将这样的二叉树称为平衡二叉树(Balanced Binary Tree)，简称平衡树。定义结点左子树与右子树的高度差为该结点的平衡因子，则平衡二叉树结点的平衡因子的值只可能是-1、0或1。因此，平衡二叉树可定义为或者是一棵空树，或者是具有下列性质的二叉树：它的左子树和右子树都是平衡二叉树，且左子树和右子树的高度差的绝对值不超过1。

25. 哈夫曼树和哈夫曼编码：

1. 哈夫曼树的定义

在许多应用中，树中结点常常被赋予一个表示某种意义的数值，称为该结点的权。从树的根到任意结点的路径长度（经过的边数）与该结点上权值的乘积，称为该结点的带权路径长度。树中所有叶结点的带权路径长度之和称为该树的带权路径长度，记为

$$WPL = \sum_{i=1}^n w_i l_i$$

式中， w_i 是第*i*个叶结点所带的权值， l_i 是该叶结点到根结点的路径长度。

在含有*n*个带权叶结点的二叉树中，其中带权路径长度(WPL)最小的二叉树称为哈夫曼树，也称最优二叉树。例如，图 5.34 中的 3 棵二叉树都有 4 个叶子结点 *a, b, c, d*，分别带权 7, 5, 2, 4，它们的带权路径长度分别为

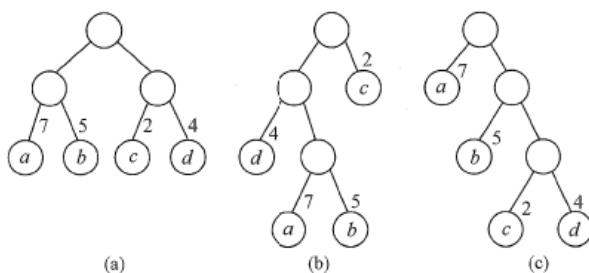


图 5.34 具有不同带权长度的二叉树

(a) $WPL = 7 \times 2 + 5 \times 2 + 2 \times 2 + 4 \times 2 = 36$ 。

(b) $WPL = 4 \times 2 + 7 \times 3 + 5 \times 3 + 2 \times 1 = 46$ 。

(c) $WPL = 7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 3 = 35$ 。

其中，图 5.34(c)树的 WPL 最小。可以验证，它恰好为哈夫曼树。

2. 哈夫曼树的构造

给定*n*个权值分别为 $w_1, w_2, \dots, w_n$ 的结点，构造哈夫曼树的算法描述如下：

- 1) 将这*n*个结点分别作为*n*棵仅含一个结点的二叉树，构成森林*F*。
- 2) 构造一个新结点，从*F*中选取两棵根结点权值最小的树作为新结点的左、右子树，并且将新结点的权值置为左、右子树上根结点的权值之和。
- 3) 从*F*中删除刚才选出的两棵树，同时将新得到的树加入*F*中。
- 4) 重复步骤2)和3)直至*F*中只剩下一棵树为止。

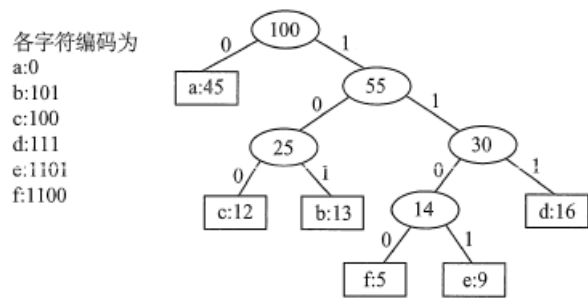
从上述构造过程中可以看出哈夫曼树具有如下特点：

- 1) 每个初始结点最终都成为叶结点，且权值越小的结点到根结点的路径长度越大。
- 2) 构造过程中共新建了*n* - 1 个结点（双分支结点），因此哈夫曼树的结点总数为2*n* - 1。
- 3) 每次构造都选择2棵树作为新结点的孩子，因此哈夫曼树中不存在度为1的结点。

3. 哈夫曼编码：

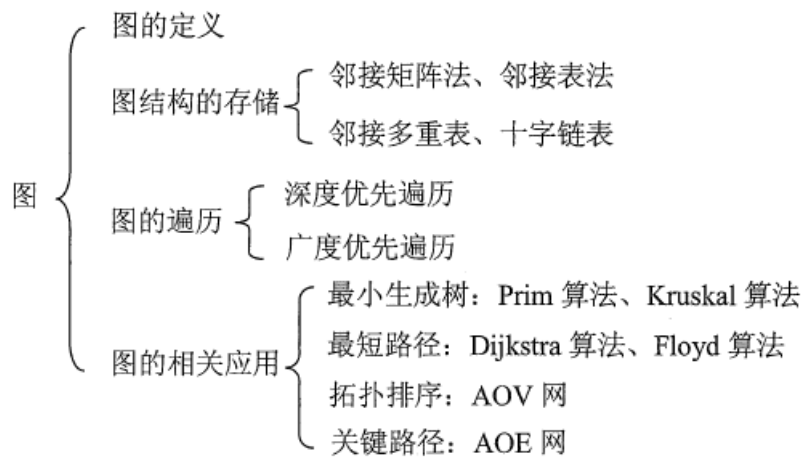
在数据通信中，若对每个字符用相等长度的二进制位表示，称这种编码方式为固定长度编码。若允许对不同字符用不等长的二进制位表示，则这种编码方式称为可变长度编码。可变长度编码比固定长度编码要好得多，其特点是对频率高的字符赋以短编码，而对频率较低的字符则赋以较长一些的编码，从而可以使字符的平均编码长度减短，起到压缩数据的效果。哈夫曼编码是一种被广泛应用而且非常有效的数据压缩编码。若没有一个编码是另一个编码的前缀，则称这样的编码为前缀编码。

由哈夫曼树得到哈夫曼编码是很自然的过程。首先，将每个出现的字符当作一个独立的结点，其权值为它出现的频度（或次数），构造出对应的哈夫曼树。显然，所有字符结点都出现在叶结点中。我们可将字符的编码解释为从根至该字符的路径上边标记的序列，其中边标记为0表示“转向左孩子”，标记为1表示“转向右孩子”。



第六章、图

快速唤起记忆知识框架：



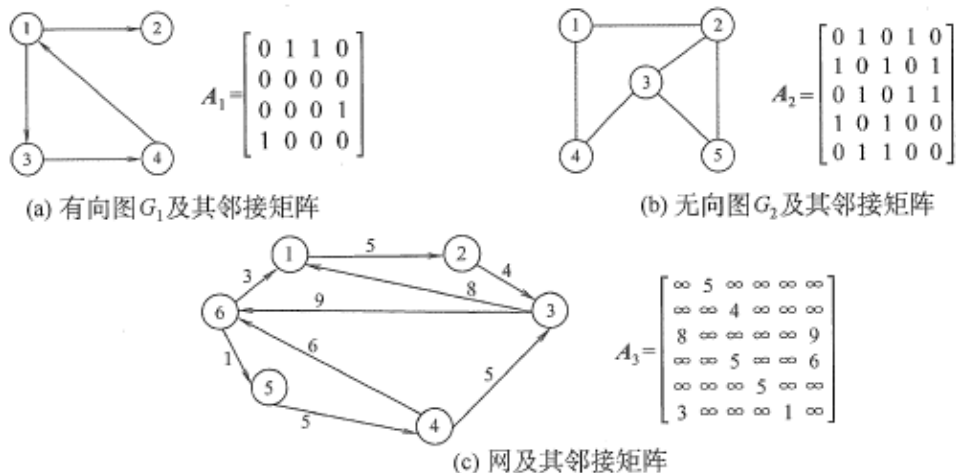
26.图的一些相关定义

这个比较基础，忘了的自己去看看把，学过离散的应该比较熟悉。

27.图的存储结构:

1.邻接矩阵法:

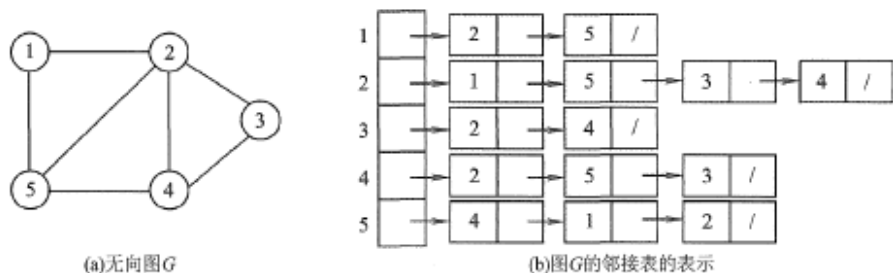
所谓邻接矩阵存储，是指用一个一维数组存储图中顶点的信息，用一个二维数组存储图中边的信息（即各顶点之间的邻接关系），存储顶点之间邻接关系的二维数组称为邻接矩阵。有向图、无向图和网对应的邻接矩阵实例图如下：



适合稠密图。

2. 邻接表法:

当一个图为稀疏图时, 使用邻接矩阵法显然要浪费大量的存储空间, 而图的邻接表法结合了顺序存储和链式存储方法, 大大减少了这种不必要的浪费。所谓邻接表, 是指对图 G 中的每个顶点 V 建立一个单链表, 第 i 个单链表中的结点表示依附于顶点 v_i 的边 (对于有向图则是以顶点 v_i 为尾的弧), 这个单链表就称为顶点 v_i 的边表 (对于有向图则称为出边表)。边表的头指针和顶点的数据信息采用顺序存储 (称为顶点表), 所以在邻接表中存在两种结点: 顶点表结点和边表结点。



3. 十字链表法:

十字链表法是有向图的一种链式存储结构。在十字链表中, 对应于有向图中的每条弧有一个结点, 对应于每个顶点也有一个结点。

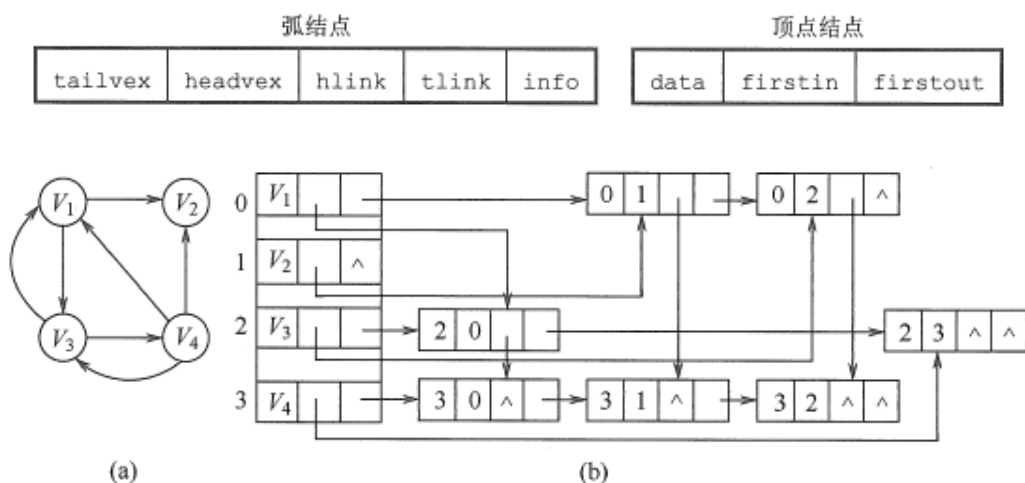


图 6.9 有向图的十字链表表示

4. 邻接多重表:

邻接多重表是无向图的另一种链式存储结构。

在邻接表中，容易求得顶点和边的各种信息，但在邻接表中求两个顶点之间是否存在边而对边执行删除等操作时，需要分别在两个顶点的边表中遍历，效率较低。与十字链表类似，在邻接多重表中，每条边用一个结点表示，每个顶点也用一个结点表示。

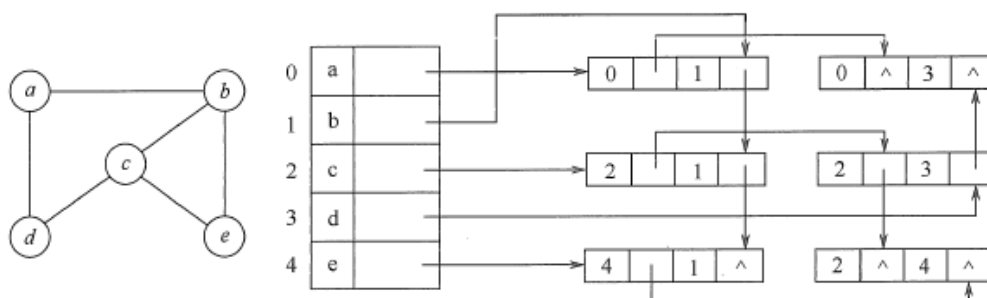
mark	ivex	ilink	jvex	jlink	info
------	------	-------	------	-------	------

其中，mark 为标志域，可用以标记该条边是否被搜索过；ivex 和 jvex 为该边依附的两个顶点在图中的位置；ilink 指向下一条依附于顶点 ivex 的边；jlink 指向下一条依附于顶点 jvex 的边，info 为指向和边相关的各种信息的指针域。

每个顶点也用一个结点表示，它由如下所示的两个域组成。

data	firstedge
------	-----------

其中，data 域存储该顶点的相关信息，firstedge 域指示第一条依附于该顶点的边。



28.图的遍历(代码请自己写)

图的遍历是指从图中的某一顶点出发，按照某种搜索方法沿着图中的边对图中的所有顶点访问一次且仅访问一次。注意到树是一种特殊的图，所以树的遍历实际上也可视为一种特殊的图的遍历。图的遍历算法是求解图的连通性问题、拓扑排序和求关键路径等算法的基础。图的遍历比树的遍历要复杂得多，因为图的任一顶点都可能和其余的顶点相邻接，所以在访问某个顶点后，可能沿着某条路径搜索又回到该顶点上。为避免同一顶点被访问多次，在遍历图的过程中，必须记下每个已访问过的顶点，为此可以设一个辅助数组visited[]来标记顶点是否被访问过。图的遍历算法主要有两种：广度优先搜索和深度优先搜索。

1.广度优先搜索(Breadth-First-Search, BFS):

类似于二叉树的层序遍历算法。基本思想是：首先访问起始顶点V，接着由V出发，依次访问V的各个未访问过的邻接顶点W₁, W₂, ..., W_n，然后依次访问W₁, W₂, ..., W_n的所有未被访问过的邻接顶点；再从这些访问过的顶点出发，访问它们所有未被访问过的邻接顶点，直至图中所有顶点都被访问过为止。若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作为初始点，重复上述过程。Dijkstra源最短路径算法和Prim最小生成树算法也应用了类似的思想。

2.深度优先搜索 (Depth-First-Search, DFS):

它的基本思想如下：首先访问图中某一起始顶点V，然后由V出发，访问与V邻接且未被访问的任一顶点W₁，再访问与W₁邻接且未被访问的任一顶点W₂.....重复上述过程。当不能再继续向下访问时，依次退回到最近被访问的顶点，若它还有邻接顶点未被访问过，则从该点开始继续上述搜索过程，直至图中所有顶点均被访问过为止。

29.最小生成树和最短路径:

这四个算法请自己去翻书或百度，这是图的比较经典的应用。

迪杰斯特拉 (dijkstra) 算法：迪杰斯特拉算法是经典的单源最短路径算法，用于求某一顶点到其他顶点的最短路径，它的特点是以起始点为中心层层向外扩展，直到扩展的终点为止，迪杰斯特拉算法要求边的权值不能为负权。

弗洛伊德 (Floyd) 算法：弗洛伊德算法是经典的求任意顶点之间的最短路径，其边的权值可为负权，该算法的时间复杂度为 $O(N^3)$ ，空间复杂度为 $O(N^2)$ 。

普里姆 (prim) 算法：用来求最小生成树，其基本思想为：从联通网络 $N=\{V,E\}$ 中某一顶点 u_0 出发，选择与他关联的最小权值的边，将其顶点加入到顶点集 S 中，此后就从一个顶点在 S 集中，另一个顶点不在 S 集中的所有顶点中选择出权值最小的边，把对应顶点加入到 S 集中，直到所有的顶点都加入到 S 集中为止。

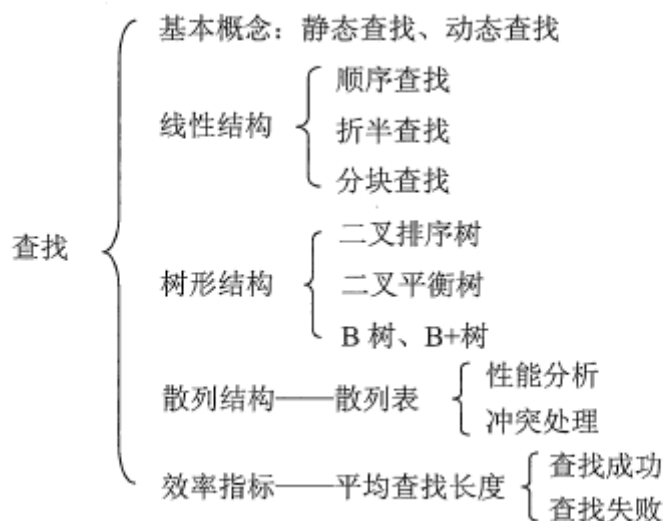
克鲁斯卡尔 (kruskal) 算法：用来求最小生成树，其基本思想为：没有一个有 N 个顶点的联通网络 $N=\{V,E\}$ ，初试时建立一个只有 N 个顶点，没有边的非连通图 T ， T 中每个顶点都看作是一个联通分支，从边集 E 中选择出权值最小的边且该边的两个端点不在一个联通分支中，则把该边加入到 T 中，否则就再从新选择一条权值最小的边，直到所有的顶点都在一个联通分支中为止。

30. 关键路径：

AOE和AOV偏理解以及手动模拟，也自己去看吧。

第七章、查找

快速唤起记忆知识框架：



31. 对各种查找方法的概括？

查找分为静态查找表和动态查找表；静态查找表包括：顺序查找、折半查找、分块查找；动态查找表包括：二叉排序树和平衡二叉树。

(1) 顺序查找：把待查关键字 key 放入哨兵位置 ($i=0$)，再从后往前依次把表中元素和 key 比较，如果返回值为0则查找失败，表中没有这个 key 值，如果返回值为元素的位置 i ($i!=0$) 则查找成功，设置哨兵的位置是为了加快执行速度。他的时间效率为 $O(n)$ ，其特点是：结构简单，对顺序结构和连式结构都适用，但是查找效率太低。

(2) 折半查找：要求查找表为顺序存储结构并且有序，若关键字在表中则返回关键字的位置，若关键字不在表中时停止查找的典型标志是：查找范围的上界 $\leq$ 查找范围的下界。

(3) 分块查找：先把查找表分为若干子表，要求每个子表的元素都要比他后面的子表的元素小，也就是保证块间是有序的（但是子表内不一定有序），把各子表中的最大关键字构成一张索引表，表中还包含各子表的起始地址。他的特点是：块间有序，块内无序，查找时块间进行索引查找，块内进行顺序查找。

(4) 二叉排序树：二叉排序树的定义为：或者是一棵空树，或者是一棵具有如下特点的树：如果该树有左子树，则其左子树的所有节点值小于根的值；若该树有右子树，则其右子树的所有节点值均大于根的值；其左右子树也分别为二叉排序树。在查找时可以进行动态的插入，插入节点要符合二叉排序树的定义，这也是动态查找和静态查找的区别，静态查找不能进行动态插入。

(5) 平衡二叉树：平衡二叉树又称为AVL树，它或者是一棵空树或者具有如下特点：他的左子树和右子树的高度差的绝对值不能大于1，且他的左右子树也都是平衡二叉树。

平衡因子：是指左子树的高度减去右子树的高度，它的值只能为1,0, -1

如果再一个平衡二叉树中插入一个节点可能造成失衡，这时就要进行树结构的调整，即平衡旋转。包括4中情况：在左子树的左子树上插入节点时向右进行单向旋转；在右子树的右子树上插入节点时向左进行单向旋转；在左子树的右子树插入节点时先向左旋转再向右旋转；在右子树的左子树插入节点时先向右旋转再向左旋转。

32.B树和B+树：

1.B 树，又称多路平衡查找树，B 树中所有结点的孩子个数的最大值称为B 树的阶，通常用m表示。一棵m 阶B 树或为空树，或为满足如下特性的m 叉树：

- 1) 树中每个结点至多有m 棵子树，即至多含有m-1 个关键字。
- 2) 若根结点不是终端结点，则至少有两棵子树。
- 3) 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字。
- 4) 所有的叶结点都出现在同一层次上，并且不带信息（可以视为外部结点或类似干折半查找判定树的查找失败结点，实际上这些结点不存在，指向这些结点的指针为空）。

B 树是所有结点的平衡因子均等于0 的多路平衡查找树。



图 7.4 一棵 5 阶 B 树的实例

2.B+树是应数据库所需而出现的一种B 树的变形树。

一棵m 阶的B+树需满足下列条件：

- 1) 每个分支结点最多有m 棵子树（孩子结点）。
- 2) 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。
- 3) 结点的子树个数与关键字个数相等。
- 4) 所有叶结点包含全部关键字及指向相应记录的指针，叶结点中将关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接起来。
- 5) 所有分支结点（可视为索引的索引）中仅包含它的各个子结点（即下一级的索引块）中关键字的最大值及指向其子结点的指针。

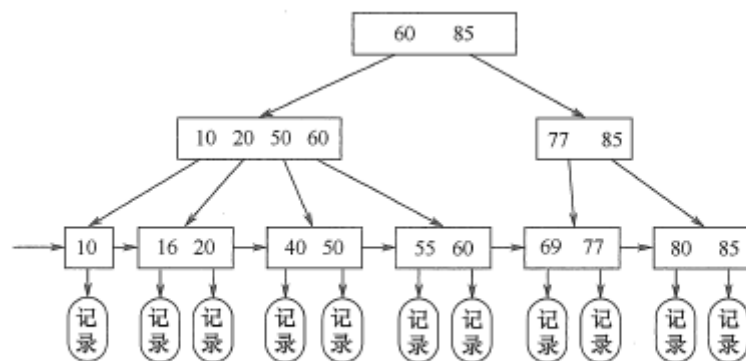


图 7.8 B+树结构示意图

m 阶的B+树与m 阶的B 树的主要差异如下：

- 1) 在B+树中，具有n 个关键字的结点只含有n 棵子树，即每个关键字对应一棵子树；而在B 树中，具有n 个关键字的结点含有n+1棵子树。
- 2) 在B+树中，每个结点（非根内部结点）的关键字数n 的范围是 $\lceil m/2 \rceil \leq n \leq m$ （根结点： $1 \leq n \leq m$ ）；在B 树中，每个结点（非根内部结点）的关键字数n 的范围是 $\lceil m/2 \rceil - 1 \leq n \leq m - 1$ 。
- 3) 在B+树中，叶结点包含信息，所有非叶结点仅起索引作用，非叶结点中的每个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有该关键字对应记录的存储地址。
- 4) 在B+树中，叶结点包含了全部关键字，即在非叶结点中出现的关键字也会出现在叶结点中；而在B 树中，叶结点包含的关键字和其他结点包含的关键字是不重复的。

32. 哈希表的概念、哈希函数的构造方法、哈希冲突的解决办法？

哈希表又称为散列表，是根据关键字码的值直接进行访问的数据结构，即它通过把关键码的值映射到表中的一个位置以加快查找速度，其中映射函数叫做**散列函数**，存放记录的数组叫做**散列表**。

哈希函数的构造方法包括：**直接定址法**，**除留余数法**，**数字分析法**，平方取中法，折叠法，随机数法

- (1) 直接定址法：取关键字的某个线性函数值作为散列地址， $H(\text{key}) = a * \text{key} + b$ 。
- (2) 除留余数法：取关键字对p取余的值作为散列地址，其中 $p < m$ ，即 $H(\text{key}) = \text{key} \% p$ ($p < m$)。
- (3) 数字分析法：当关键字的位数大于地址的位数，对关键字的各位分布进行分析，选出分布均匀的任意几位作为散列的地址，适用于所有关键字都已知的情况。
- (4) 平方取中法：对关键字求平方，再取结果中的中间几位作为散列地址。
- (5) 折叠法：将关键字分为位数相同的几部分，然后取这几部分的叠加和作为散列地址。适用于关键字位数较多，且关键字中每一位上数字分布大致均匀。
- (6) 随机数法：选择一个随机函数，把关键字的随机函数值作为散列地址。适合于关键字的长度不相同。

哈希冲突的解决方法包括：**开放定址法**和**拉链法**，当冲突发生时，使用某种探测技术形成一个探测序列，然后沿此序列逐个单元查找，直到找到该关键字或者碰到一个开放的地址为止，探测到开放的地址表明该表中没有此关键字，若要插入，则探测到开放地址时可将新节点插入该地址单元。其中开放定址法包括：线性探查法，二次探查法，双重散列法

- (1) **线性探查法**：基本思想，探查时从地址d开始，首先探查 $T[d]$ ，在探查 $T[d+1]$...直到查到 $T[m-1]$ ，此后循环到 $T[0]$ ， $T[1]$...直到探测到 $T[d-1]$ 为止。

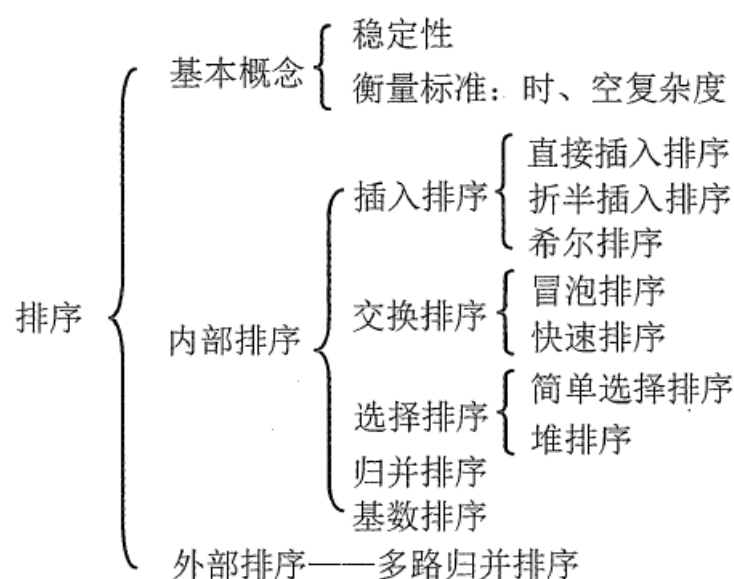
(2) **二次探查法**：基本思想，探查时从地址d开始，首先探查T[d],再探查T[d+1²],T[d+2²]...等，直到探查到有空余地址或者探查到T[d-1]为止，缺点是无法探查到整个散列空间。

(3) 双重散列法：基本思想，使用两个散列函数来确定地址，探查时从地址d开始，首先探查T[d],再探查T[d+h₁(d)],T[d+2*h₁(d)]...

链接法：将所有关键字为同义词的节点链接在同一个单链表中，若选定的散列表长度为m，则可将散列表定义为一个由m个头指针组成的指针数组，凡是散列地址为i的节点均插入到头指针为i的单链表中。

第八章、排序

快速唤起记忆知识框架：



33.对各种内部排序的概括与总结？

排序：是指把一个任意元素的序列排列成一个按关键字key有序的序列。内部排序包括：插入排序、选择排序、交换排序、归并排序、基数排序。其中插入排序包括：直接插入排序、折半插入排序、希尔排序；选择排序包括：简单选择排序，堆排序；交换排序包括：冒泡排序、快速排序。

(1) 直接插入排序（稳定）：基本思想为：将序列分为有序部分和无序部分，从无序部分依次选择元素与有序部分比较找到合适的位置，将原来的元素往后移，将元素插入到相应位置上。时间复杂度为： $O(n^2)$ ，空间复杂度为 $O(1)$

(2) 折半插入排序（稳定）：基本思想为：设置三个变量low high mid，令 $mid=(low+high)/2$ ，若 $a[mid]>key$ ，则令 $high=mid-1$ ，否则令 $low=mid+1$ ，直到 $low>high$ 时停止循环，对序列中的每个元素做以上处理，找到合适位置将其他元素后移进行插入。他的比较次数为 $O(n\log_2 n)$ ，但是因为要后移，因此时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(1)$ 。优点是：比较次数大大减少。

(3) **希尔排序**（不稳定）：基本思想为：**先将序列分为若干个子序列，对各个子序列进行直接插入排序，等到序列基本有序时再对整个序列进行一次直接插入排序。**优点是：让关键字值小的元素能够很快移动到前面，且**序列基本有序时进行直接插入排序时间效率会提升很多**，空间复杂度为 $O(1)$ 。

(4) 简单选择排序（不稳定）：基本思想为：将序列分为2部分，每经过一趟就在无序部分找到一个最小值然后与无序部分的第一个元素交换位置。优点是：实现起来特别简单，缺点是：每一趟只能确定一个元素的位置，时间效率低。时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(1)$ 。

(5) 堆排序（不稳定）：设有一个任意序列， $k_1, k_2, \dots, k_n$ ，当满足下面特点时称之为堆：让此序列排列成完全二叉树，该树具有以下特点，该树中任意节点均大于或小于其左右孩子，此树的根节点为最大值或者最小值。优点是：对大文件效率明显提高，但对小文件效率不明显。时间复杂度为 $O(n \log_2 n)$ ，空间复杂度为 $O(1)$ 。

(6) 冒泡排序（稳定）：基本思路为：每一趟都将元素进行两两比较，并且按照“前小后大”的规则进行交换。优点是：每一趟不仅能找到一个最大的元素放到序列后面，而且还把其他元素理顺，如果下一趟排序没有发生交换则可以提前结束排序。时间复杂度为 $O(n^2)$ ，空间复杂度为 $O(1)$ 。

(7) 快速排序（不稳定）：基本思路为：在序列中任意选择一个元素作为中心，比它大的元素一律向后移动，比它小的元素一律向前移动，形成左右两个子序列，再把子序列按上述操作进行调整，直到所有的子序列中都只有一个元素时序列即为有序。优点是：每一趟不仅能确定一个元素，时间效率较高。时间复杂度为 $O(n \log_2 n)$ ，空间复杂度为 $O(\log_2 n)$ 。

(8) 归并排序（稳定）：基本思想为：把两个或者两个以上的有序表合并成一个新的有序表。时间复杂度为 $O(n \log n)$ ，空间复杂度和待排序的元素个数相同。

(9) 基数排序：时间复杂度为：对于 n 个记录进行链式基数排序的时间复杂度为 $O(d(n+rd))$ ，其中每一趟分配的时间复杂度为 $O(n)$ ，回收的时间复杂度为 $O(rd)$ 。

各种排序的总结表格如下：

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	否
2路归并排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

直接插入排序、冒泡排序和简单选择排序是基本的排序方法，它们主要用于元素个数 n 不是很大($n < 10000$)的情形。

对于中等规模的元素序列($n \leq 1000$)，希尔排序是一种很好的选择。

对于元素个数 n 很大的情况，可以采用快排、堆排序、归并排序或基数排序，其中快排和堆排序都是不稳定的，而归并排序和基数排序是稳定的排序算法。

